
Asterion: Deterministic Trading-System Infrastructure Under Latency and Allocation Constraints

Anannye Naik

Abstract

Trading-system research often evaluates algorithms separately from the infrastructure that converts market events into executable order flow. We introduce Asterion, a deterministic C++20 systems platform that joins fixed-record replay, L3 order-book maintenance, price-time matching, pre-trade risk, execution reporting and model inference under common validation and measurement. Deterministic checksums and an independent Python reference matcher provide controls for comparing implementation variants after their tested outputs agree. On Durham University's Hamilton8 cluster, a timed path through replay, L3/L2 book updates, strategy and risk processes 5.0×10^6 measured high-cancellation events at p50 281 ns, p99 631 ns and 2.88×10^6 events/s, with 7.34×10^6 measured allocations. An opt-in pooled book removes all measured allocations and preserves the same guard checksum, but p99 rises to 681 ns. Across eight stress corpora, pooled storage reduces median latency in every case but does not uniformly reduce the tail. A bounded SPSC pipeline preserves checksum parity with zero drops across six 10^6 -event corpora while throughput ranges from $0.70\times$ to $1.23\times$ of single-thread replay. In-loop LinearModel inference adds 140 ns at the median with no additional measured allocations; a separate ONNX Runtime contract costs p50 36.9 μ s over 20,000 calls. These results show that memory, concurrency and inference optimisations require output-controlled evaluation rather than architectural intuition alone.

1. Introduction

Trading-system latency belongs to the path that turns an event into an order decision and, when applicable, an execution report. Parsing, book updates, feature production, strategy decisions, risk checks and matching can each alter its cost. Measuring one operation in isolation therefore misses interactions introduced by data structures, schedul-

ing and model calls. A systems experiment must define both the event path and the boundary at which time is measured.

Comparing paths also requires a behavioural control. A faster variant that changes order priority, loses replay events or alters risk decisions is a different system. Checksums can expose output differences for fixed inputs, while independent semantics and targeted tests examine behaviours that a checksum alone cannot establish. These controls make latency and allocation differences interpretable as implementation effects within the tested scope.

We introduce Asterion, a C++20 platform connecting fixed-record event replay to individual-order L3 state, aggregate L2 views, strategy and inference decisions, pre-trade risk, price-time matching and structured execution reports. Prices remain integer ticks throughout, and deterministic input and output surfaces are exposed along the path. Figure 1 shows the architecture and the order of validation and measurement. The headline timed loop includes replay, L3/L2 updates, strategy and risk; matching and report lifecycle are implemented and checked through separate semantic tests.

Four measurement boundaries isolate different costs. A standard node-based book is compared with an opt-in pooled book under the same benchmark guard. Sequential replay is compared with a bounded, lossless single-producer/single-consumer (SPSC) replay. A synchronous LinearModel stage is inserted into the Hamilton8 event loop, while a recorded-public-L2 ONNX artefact is scored through an isolated runtime contract on another host. The last boundary times an inference call rather than the replay loop.

The interactions are uneven. Pooled storage eliminates measured allocation but can raise high-cancellation tail latency. The SPSC path improves throughput for high cancellation and reduces it on five other corpora. In-loop LinearModel cost exceeds the isolated model-call median, and runtime-backed ONNX inference occupies a different time scale and environment.

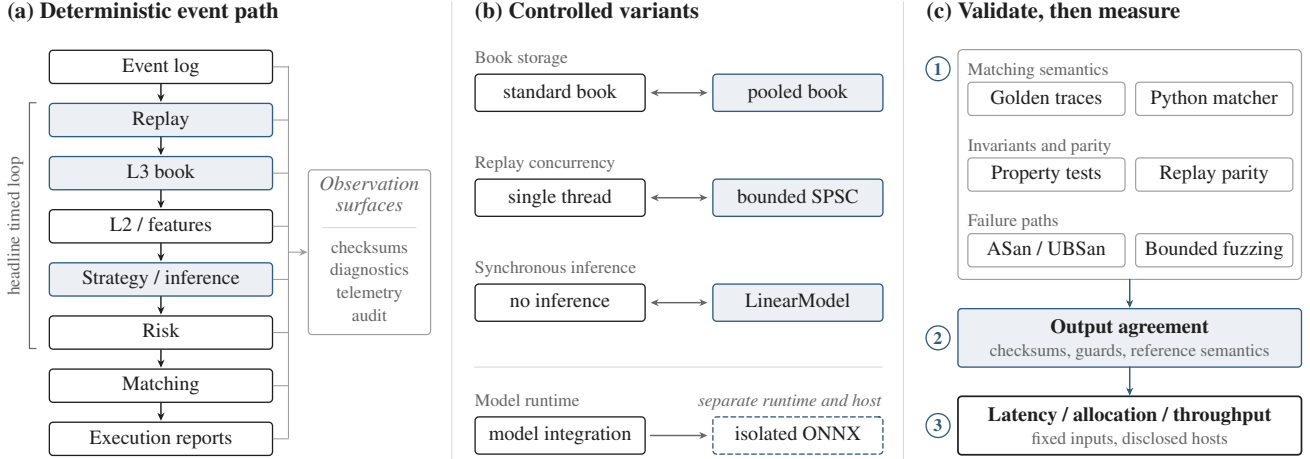


Figure 1. Experimental design. (a) Deterministic path from fixed-record replay to execution reports. Shaded stages are varied in (b); the bracket marks the headline timed loop; observation surfaces sit outside the data path. (b) Each comparison changes one component on identical inputs; the ONNX Runtime contract is an isolated call on a separate host. (c) Semantic, replay, sanitizer and fuzzing checks establish output agreement before measurement, so differences can be read as implementation effects within the tested scope.

We make three contributions:

- A deterministic replay-to-execution architecture combining L3 state, price-time matching, pre-trade risk, execution reports and model integration within one C++20 event path.
- An output-controlled validation methodology combining deterministic checksums with independent matching semantics, replay parity, property testing, sanitizers and fuzzing before performance variants are compared.
- A measured systems study showing that allocation removal, SPSC concurrency and inference integration have distinct workload and percentile effects under disclosed Hamilton8 and ONNX Runtime environments.

2. Related Work

Individual orders and queue position are central to limit-order-book structure (Gould et al., 2013). Event-level additions, executions and cancellations can be related to changes in displayed depth and short-term price formation (Cont et al., 2014). Nasdaq’s TotalView–ITCH specification describes an order-level market-data feed (Nasdaq, n.d.); LOBSTER reconstructs book histories from its historical ITCH messages (Huang & Polak, 2011). Asterion’s ASTITCH1 is an internal fixed-record schema for deterministic replay, distinct from Nasdaq’s wire format.

ABIDES provides agent-based discrete-event market simulation with an exchange agent and configurable message latency (Byrd et al., 2019). Liquibook supplies C++ order-matching components and optional depth-book maintenance (Liquibook contributors, n.d.). JAX-LOB uses GPU-parallel

book simulation for large-scale reinforcement learning (Frey et al., 2023). These systems establish different matching and simulation boundaries; the present CPU experiment measures resource changes between tested Asterion variants under output agreement.

Execution reports encode order status, fills and remaining quantity in the FIX order lifecycle (FIX Trading Community, 2023). Asterion uses an internal structured report type and report-driven working-order updates, drawing on lifecycle semantics without adopting FIX wire encoding. Its pre-trade controls are evaluated against the implemented order and market-data state.

FastForward studies cache-optimised SPSC communication for pipeline parallelism (Giacomoni et al., 2008); Asterion measures a bounded queue inside replay across six workloads. AddressSanitizer detects memory-access errors through compiler instrumentation (Serebryany et al., 2012). Here sanitizer builds and bounded fuzzing complement reference matching and replay parity before timing.

ONNX Runtime provides a separate model execution boundary through a CPU execution provider (ONNX Runtime contributors, n.d.). The recorded-public-L2 artefact originates from the ChronosLOB repository (Naik, 2026); the present study measures the runtime contract through which Asterion scores it.

3. System Design and Experimental Controls

3.1. Event Replay and Book State

Asterion represents replay input through the fixed `asterion.event_log v1` schema, available in CSV and

binary form. Its binary encoding uses `ASTITCH1` magic, a 16-byte header and 58-byte little-endian fixed records. Records hold integer timestamp, sequence, symbol, event type, side, flags, price ticks, quantity and order/trade identifiers. CSV and binary readers map to the same logical events and share an event checksum for identical content. Replay records sequence and timestamp diagnostics, a diagnostics checksum and a final-book checksum; these surfaces help distinguish an input difference from a state difference.

The L3 book retains individual order identity and FIFO order within each price level. Adds, cancels, reductions and replacements update this state, and aggregate L2 depth is derived from it. Price and quantity remain integral in replay and matching. The pooled variant pre-reserves order and level storage and reuses nodes without changing the tested book behaviour. Its zero measured allocations refer to the configured hot-path runs.

3.2. Matching, Risk and Execution

Matching processes limit and market requests against best opposing price levels in price-time order. Good-till-cancelled (GTC) residual limit quantity can rest; immediate-or-cancel (IOC) residuals are cancelled; fill-or-kill (FOK) requires enough executable quantity before matching. A post-only request that would cross is rejected. Cancellation removes a resting order, while replacement cancels and reinserts the remaining order at the new price and timestamp, losing its old FIFO position. The implemented incoming-order self-trade policy rejects an attributed incoming request if it would cross that client's resting opposite-side order. The rejection is preventive: no trade is cancelled after a fill.

Pre-trade risk checks order quantity, tick notional, per-symbol position, gross exposure, a price band and stale market data. Tick notional is absolute price ticks multiplied by quantity; market requests use the current reference price. Optional controls track working-order quantity, message rate and self-trade exposure. Connection and kill state can reject new flow; configured disconnect or kill handling releases simulated working exposure. Execution reports update outstanding quantity after acceptance, trade, cancellation, replacement or rejection. An opt-in audit trail records decisions and state changes. All of these controls are implemented and exercised in tests; audit recording is disabled in the measured hot path.

3.3. Implementation Variants

The standard and pooled book variants process the same high-cancellation input with a shared guard checksum. Stress corpora vary cancellation, replacement, depth, burstiness, price range, symbol concentration and lifecycle patterns. The SPSC path passes fixed-size events through a 4096-slot ring buffer; the measured configuration blocks

the producer on a full queue rather than shedding events. LinearModel inference uses a caller-owned feature buffer inside the sequential event loop. Its score is folded into the guard so the model stage remains observable, but it does not change the strategy or risk decision in that benchmark. The ONNX measurement times an isolated C++ runtime call for a model contract that sits outside the replay loop.

3.4. Correctness Controls

Fixed-input event, diagnostics, book, report and benchmark-guard checksums expose deterministic output surfaces. Golden traces and the independent Python reference matcher check matching semantics. Fixed-seed property and random tests exercise book and order lifecycle invariants; grouped and shared replay parity checks multi-symbol handling. AddressSanitizer and UndefinedBehaviorSanitizer builds, plus bounded libFuzzer targets for parsing, replay, matching requests and audit input, exercise additional failure paths. These checks establish agreement on the tested surfaces and detect classes of errors, but they do not prove implementation equivalence.

4. Experimental Design

Deterministic synthetic corpora underpin the Hamilton8 measurements; a corpus manifest records their seeds and SHA-256 digests. The headline high-cancellation loop processes binary replay, book updates, L2 production, strategy and risk. Book variants use the same corpus and guard. The inference-loop comparison uses balanced 10^4 -event input and the standard book. SPSC throughput compares single-thread and steady-state producer/consumer replay on six 10^6 -event inputs. Table 1 records build and measurement conditions.

Each benchmark row is one measured run. Reported latency percentiles describe within-run event variation rather than between-run confidence intervals. Throughput counts consumed or processed events over the row's measured interval; allocation counts come from the project's allocation tracker. The Windows ONNX measurement characterises the inference contract alone and is not placed on a common timing axis with Hamilton8 data.

5. Results

5.1. Allocation Removal Under Output Agreement

All eight paired stress-corpus rows report zero measured pooled allocations and true guard parity. Pooled p50 improves in every row by 20–70 ns; p99 improves in seven. The wide-price-range row is the exception: p50 falls from 491 to 421 ns, while p99 rises from 772 to 882 ns, a 110 ns regression. Figure 2 shows paired latency deltas with

Table 1. Measurement protocol. Hamilton8 event-path rows share one Slurm allocation. The ONNX Runtime call uses a separate host and timing boundary.

Component	Specification
Primary host	Durham University Hamilton8; Slurm job 17356789; cn025.ham8.dur.ac.uk; AMD EPYC 7702; Rocky Linux 8.10.
Build and allocation	GCC 13.2; Release <code>-O3 -DNDEBUG</code> ; one node, one task, one CPU, 8 GiB; logical CPU 121 pinned.
Headline high cancellation	One 10^6 -event corpus; five measured iterations per book variant; 5×10^6 measured events per row.
Inference loop	Balanced 10^4 -event corpus; 50 measured iterations; 5×10^5 measured events per row.
SPSC steady state	Six 10^6 -event corpora; 4096-slot queue; blocking backpressure and measured drop/parity surfaces.
Separate ONNX contract	20,000 C++ calls; input $1 \times 16 \times 40$, output 1×3 ; Windows 10/MSYS2 UCRT64, GCC 16.1, ONNX Runtime 1.20.1 CPU execution provider.
Measurement	Warm-up before measured loops; allocation counters reset; p50, p95, p99, p99.9, throughput, allocation counts and output checks.

Table 2. Hamilton8 headline measurements. Percentiles are per-event nanoseconds; throughput is events/s. The inference score is observed but does not reach order flow in this timed loop.

Path	p50	p95	p99	p99.9	Throughput	Measured allocations
Standard high cancellation	281	501	631	812	2,879,594	7,340,580
Pooled high cancellation	291	491	681	832	2,815,909	0
Standard balanced, no inference	371	511	601	831	2,132,930	703,450
Standard balanced + LinearModel	511	671	792	1,002	1,649,716	703,450

standard-to-pooled allocations per event; the full rows appear in Appendix B.

The larger high-cancellation measurement uses 5×10^6 events per variant. Standard storage records p50/p95/p99/p99.9 of 281/501/631/812 ns, 2,879,594 events/s and 7,340,580 allocations. The pooled path records 291/491/681/832 ns, 2,815,909 events/s and zero allocations. Both retain guard 14180005740461440914. Removing every measured allocation therefore does not guarantee uniformly lower latency.

5.2. Tail Behaviour and Inference Integration

On the high-cancellation row, pooling changes p50 by +10 ns, p95 by −10 ns, p99 by +50 ns and p99.9 by +20 ns. The direction changes across percentiles, so a median-only reading conceals the tail trade-off. Figure 3(a) displays the measured percentile paths; maxima are also recorded but are not used to characterise individual events.

The standard balanced loop without inference records p50/p95/p99/p99.9 of 371/511/601/831 ns, 2,132,930 events/s and 703,450 allocations. With synchronous LinearModel scoring, the values are 511/671/792/1,002 ns, 1,649,716 events/s and the same 703,450 allocations. Thus complete loop integration adds 140 ns at p50 and 191 ns at

p99 without additional measured allocation on this standard-book path. The inference guard differs because it includes the observed score; the underlying strategy and risk decision are unchanged in the benchmark.

The separate feature-extraction and linear-call rows each record p50 30 ns and p99 40 ns. A caller-owned feature buffer with measured LinearModel inference records p50 121 ns and p99 151 ns. These component medians do not sum to the 140 ns complete-loop change: timing boundaries, surrounding decisions and interactions differ. An isolated model call therefore does not determine event-loop integration cost.

5.3. Concurrency Is Workload-Dependent

The bounded SPSC path matches the single-thread replay checksum, drops zero events and reaches queue depth 4096 in every one of six 10^6 -event corpus rows. Its throughput ratios are $0.79\times$ for balanced flow, $1.23\times$ for high cancellation, $0.81\times$ for replace-heavy and deep-book flow, $0.73\times$ for bursty flow and $0.70\times$ for wide-price-range flow. Figure 4 isolates the ratio; Appendix B gives both measured throughputs and backpressure counts.

The producer and consumer introduce queue traffic and scheduling costs while shifting replay work between threads.

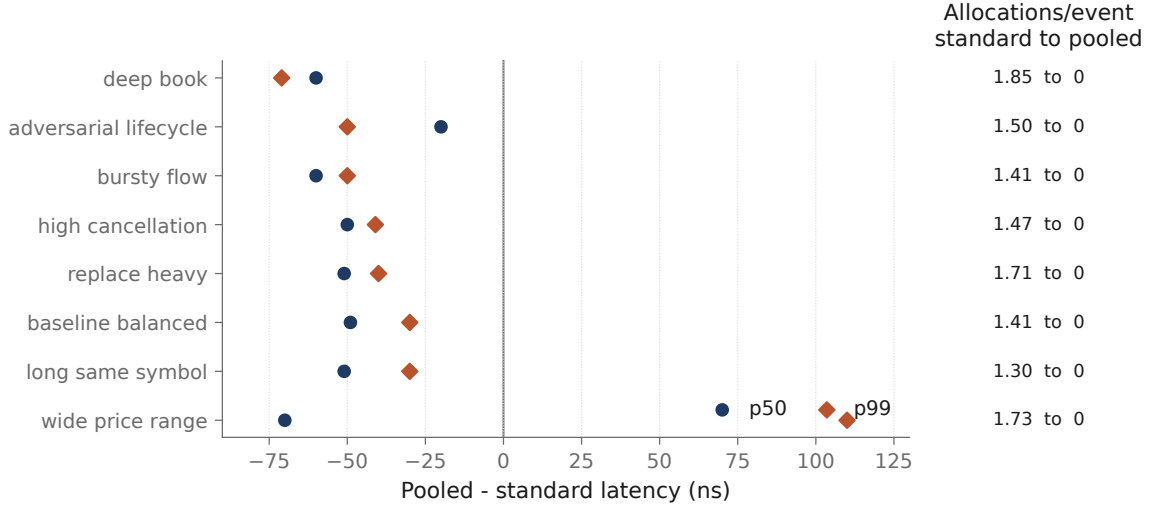


Figure 2. Pooled minus standard latency over eight stress corpora. Negative values favour pooling. Both percentile markers are from one measured row per corpus; the right column shows standard allocations per event falling to zero under guard parity.

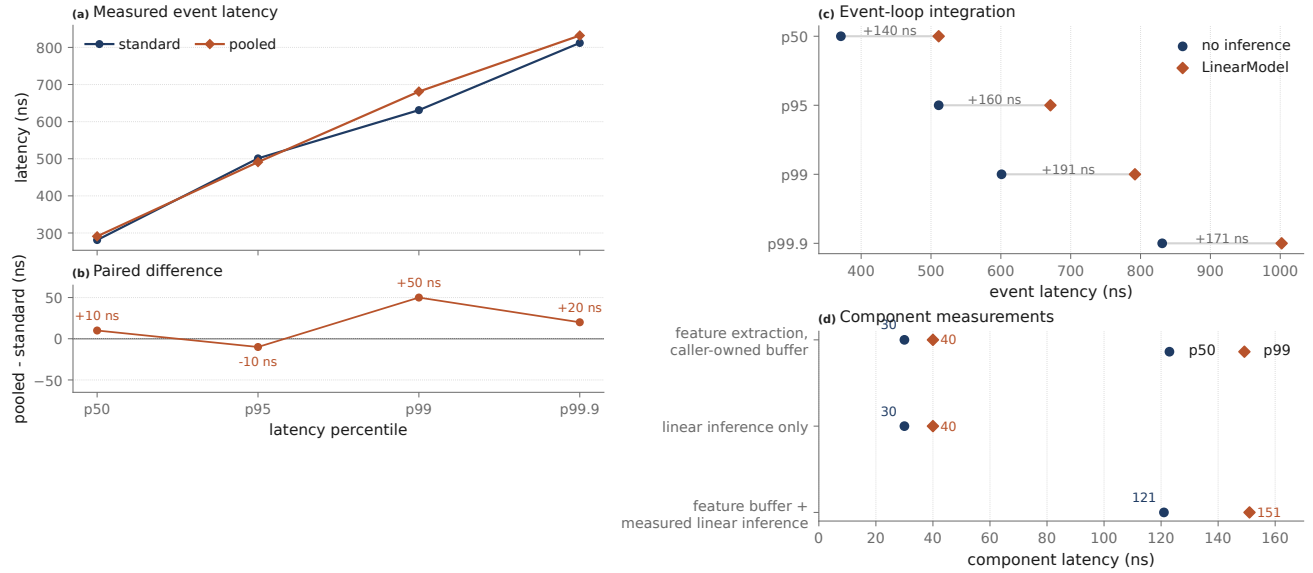


Figure 3. Four Hamilton8 timing views. (a) Standard and pooled high-cancellation percentiles. (b) Pooled minus standard differences: pooling changes p50 by +10 ns and p99 by +50 ns under identical guard output. (c) Complete-loop LinearModel integration. (d) Feature and model component measurements at their separate timing boundary.

Which effect dominates depends on the event mix. Under the measured lossless policy, concurrency is a workload-dependent throughput choice rather than an automatic improvement.

5.4. Runtime-Bound Model Contract

The recorded-public-L2 ONNX artefact has input $1 \times 16 \times 40$ and output 1×3 . On the separate Windows 10/MSYS2 UCRT64 host, 20,000 C++ calls through ONNX Runtime 1.20.1 CPU execution provider record p50/p95/p99/p99.9 of 36.9/72.3/108.9/409.7 μ s, approximately 23,200 infer-

ences/s, two measured allocations and 2568 bytes per call. Model load is a separate 3.26 ms measurement. These values are reproduced unchanged from a ChronosLOB model-bridge benchmark recorded on 2026-06-04 and were not re-measured for Asterion. They characterise a runtime-backed integration cost, not a Hamilton8 replay-path latency.

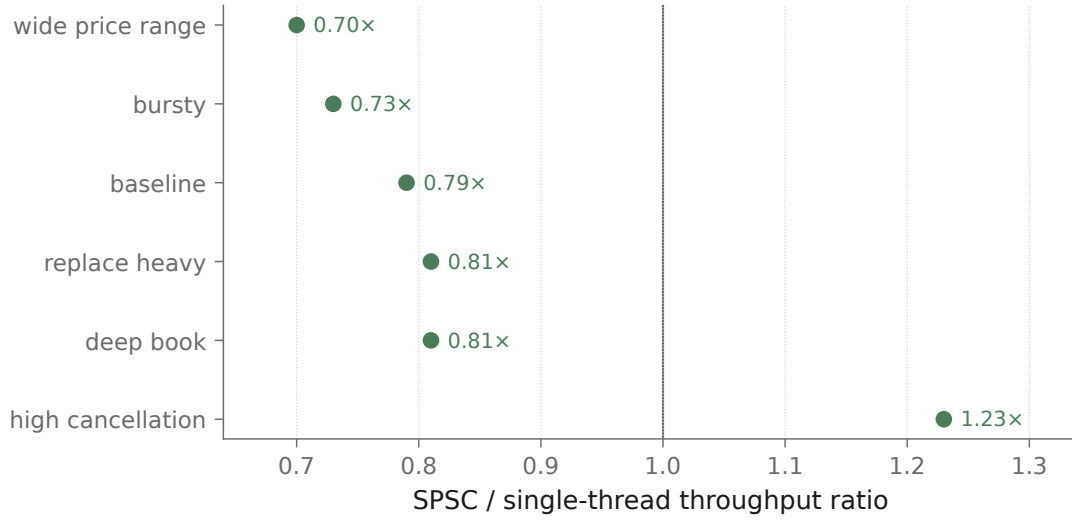


Figure 4. Steady-state SPSC throughput divided by single-thread replay throughput across six Hamilton8 corpora. The bounded blocking queue had no drops, reached capacity in every row and retained checksum parity; only high cancellation exceeds one.

6. Discussion and Validity

Allocation and latency are distinct measured effects. Pooling removes tracked steady-state allocation in every tested corpus, yet its high-cancellation p99 rises and wide-price-range p99 regresses. A reservation and reuse strategy can change lookup and memory-access costs even when its allocation count is lower. The data support the measured trade-off without identifying one microarchitectural cause.

The SPSC and inference comparisons likewise resist an architecture-only prediction. Queueing helps one event mix and hurts five; a synchronous model adds more to the event loop than its isolated call median. The ONNX measurement differs in runtime, host and boundary, so its absolute latency describes the model contract in that environment only.

The benchmark guards and replay checksums make fixed-output comparisons possible at their tested boundaries. Golden traces, reference matching and lifecycle tests probe semantics outside the timed loop. Agreement on these surfaces limits a performance comparison to paths whose observed behaviour is held fixed; it cannot establish correctness for every unseen order sequence.

The measurements are tied to the disclosed workloads and execution environments. Each row is one measured run on one shared Hamilton8 Slurm allocation, with within-run percentiles rather than repeated-run uncertainty. The non-root job did not control the CPU governor or turbo state. Most timed inputs are deterministic synthetic stress corpora; recorded public-L2 fixtures exercise a different replay and model-contract boundary. The ONNX measurements use another host. These conditions define what the reported costs compare.

7. Conclusion

Asterion treats the replay-to-execution path as a measurable systems object. Its central result is not a single latency number, but a set of controlled interactions. Pooled storage removes measured steady-state allocation while leaving tail latency workload-dependent. A bounded SPSC pipeline preserves the tested replay outputs but produces throughput ratios from 0.70x to 1.23x. Deterministic LinearModel inference adds 140 ns at the median without additional measured allocations, while a runtime-backed ONNX contract operates on a tens-of-microseconds scale.

These findings support a broader engineering principle: optimisation should be evaluated after the behaviour being optimised has been fixed and checked. In event-driven trading infrastructure, memory management, concurrency and model integration affect different parts of the cost surface. Deterministic replay, independent semantic checks and disclosed measurement conditions make those effects visible without collapsing them into a single benchmark score.

References

- Byrd, D., Hybinette, M., and Balch, T. H. ABIDES: Towards high-fidelity market simulation for AI research. arXiv:1904.12066, 2019. URL <https://arxiv.org/abs/1904.12066>.
- Cont, R., Kukanov, A., and Stoikov, S. The price impact of order book events. *Journal of Financial Econometrics*, 12(1):47–88, 2014. doi: 10.1093/jffinec/nbt003.
- FIX Trading Community. FIX Latest as of EP284: Order state changes, 2023. URL <https://www.fixtrading.org/wp-content/uploads/download-manager-files/FIX-Latest-as-of-EP284-Order-State-Changes.pdf>. November 2023.
- Frey, S. Y., Li, K., Nagy, P., Sapora, S., Lu, C., Zohren, S., Foerster, J., and Calinescu, A. JAX-LOB: A GPU-accelerated limit order book simulator to unlock large-scale reinforcement learning for trading. In *4th ACM International Conference on AI in Finance*, pp. 583–591, 2023. doi: 10.1145/3604237.3626880.
- Giacomoni, J., Moseley, T., and Vachharajani, M. FastForward for efficient pipeline parallelism: A cache-optimized concurrent lock-free queue. In *Proceedings of the 13th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pp. 43–52, 2008. doi: 10.1145/1345206.1345215.
- Gould, M. D., Porter, M. A., Williams, S., McDonald, M., Fenn, D. J., and Howison, S. D. Limit order books. *Quantitative Finance*, 13(11):1709–1742, 2013. doi: 10.1080/14697688.2013.803148.
- Huang, R. and Polak, T. LOBSTER: Limit order book reconstruction system. Technical report, Humboldt-Universität zu Berlin, 2011. SSRN 1977207; doi: 10.2139/ssrn.1977207.
- Liquibook contributors. Liquibook: Open source order matching engine, n.d. URL <https://github.com/enewhuis/liquibook>. Accessed 15 September 2026.
- Naik, A. ChronosLOB: Limit order book forecasting with execution-aware validation. Research preprint and source repository, 2026. URL <https://github.com/ananynenaik/chronos-lob>.
- Nasdaq. Nasdaq TotalView–ITCH 5.0 interface specification, n.d. URL <https://nasdaqtrader.com/content/technicalsupport/specifications/dataproducts/NQTVITCHSpecification.pdf>. Accessed 15 September 2026.
- ONNX Runtime contributors. ONNX Runtime documentation, n.d. URL <https://onnxruntime.ai/docs/>. Accessed 15 September 2026.
- Serebryany, K., Bruening, D., Potapenko, A., and Vyukov, D. AddressSanitizer: A fast address sanity checker. In *2012 USENIX Annual Technical Conference*, pp. 309–318. USENIX Association, 2012. URL <https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany>.

A. Matching and Risk Semantics

Table 3 specifies the order and risk behaviour implemented and exercised in tests. The matching book and the replay book use individual order identity and FIFO priority at a price; risk maintains its own working-order view and updates it from reports. Report checksums are deterministic for a fixed request sequence but do not replace semantic assertions.

Table 3. Implemented matching and risk semantics. Optional controls are identified explicitly; the timed headline loop checks risk decisions but does not time matching reports.

Control	Behaviour
Priority and fill	Limit and market requests consume the best executable opposing level; resting orders fill FIFO at a price. Reports record both resting and incoming trade sides.
GTC, IOC, FOK	GTC limit residuals rest; IOC residuals cancel; FOK rejects before matching when executable quantity is insufficient.
Post-only	Only a GTC limit request is eligible; a request that would cross is rejected.
Cancel and replace	Cancel removes a resting order. Replace cancels, updates price and remaining quantity, matches if crossing, then reinserts any residual at the back of its new price queue.
Self-trade prevention	An attributed incoming or replacement request crossing the same client's resting opposite side is rejected before any fill. Unattributed client ID zero bypasses this matcher check.
Risk limits	Quantity, tick notional, per-symbol position, gross exposure, price band and market-data age are checked before acceptance.
Optional risk state	Working-order quantity, fixed or sliding message rate and same-client crossing checks can be enabled. Execution reports release or update working exposure.
Connection and kill	Disconnected and kill states can reject new requests; configured disconnect and kill handling releases simulated working orders and records state.
Audit	Opt-in decision and lifecycle records expose an audit checksum and log. Audit recording is off in the measured hot path.

B. Additional Performance Results

Table 4 lists all eight paired small-corpus rows. They are one measured run per corpus, with five passes through 4000-, 5000- or 8000-event inputs. The p50 and p99 columns are event nanoseconds. Guard parity holds in every row, and the pooled allocation count is zero throughout. The wide-price-range p99 increase is the only tail regression among the stress rows.

Table 4. Standard and pooled stress rows from `results/stress_corpora_standard_vs_pooled.csv`. Guard parity holds in every row; allocations are measured totals.

Corpus	Events	Std p50	Pool p50	Std p99	Pool p99	Std alloc.	Pool alloc.
Adversarial lifecycle	20,000	200	180	361	311	30,000	0
Balanced	20,000	300	251	441	411	28,125	0
Bursty flow	20,000	341	281	511	461	28,275	0
Deep book	25,000	331	271	541	470	46,205	0
High cancellation	20,000	250	200	421	380	29,390	0
Long same symbol	40,000	321	270	471	441	51,895	0
Replace heavy	20,000	321	270	421	381	34,260	0
Wide price range	25,000	491	421	772	882	43,125	0

The high-cancellation headline measurement differs from the small stress row in scale and corpus size. Its standard and pooled guards both equal 14180005740461440914; the larger rows use five measured passes through one million events. Table 2 gives their complete percentile and throughput record.

Table 5. Lossless steady-state SPSC replay over six 10^6 -event corpora. Throughputs are events/s; the ratio is SPSC divided by single-thread. Backpressure counts events whose initial producer enqueue attempt found the queue full; spin/yield retries are excluded. All rows have true checksum parity.

Corpus	Single thread	SPSC	Ratio	Back-pressure	Drops	Max queue depth
Balanced	1,551,396	1,221,553	0.79	245	0	4096
High cancellation	3,748,307	4,627,770	1.23	244	0	4096
Replace heavy	1,690,788	1,368,047	0.81	245	0	4096
Deep book	1,007,316	816,142	0.81	246	0	4096
Bursty	1,609,860	1,170,675	0.73	246	0	4096
Wide price range	1,273,384	896,940	0.70	246	0	4096

The SPSC allocation totals are within three allocations of the corresponding single-thread totals in each row. This is separate from the pooled-book experiment: the measured SPSC rows use the standard replay book, and their throughput effect is attributed to the tested producer/consumer path.

Table 6. Selected Linux `perf stat` counters around whole benchmark processes, not individual events. Last-level-cache events were unsupported for the Hamilton8 allocation.

Process	Instructions/cycle	Branch miss (%)	L1D miss (%)
High-cancellation hot path	1.915	1.06	0.52
Balanced steady SPSC	1.003	0.80	3.19
Replace-heavy steady SPSC	1.080	0.81	2.95
Deep-book steady SPSC	0.909	1.20	4.52

Table 7. Corpus-manifest summary for the measured synthetic inputs. The corpus manifest records the full SHA-256 of each generated corpus.

Use	Modes	Events/input	Seeds
Steady SPSC	Balanced, high-cancel, replace-heavy, deep-book, bursty, wide-price-range	10^6	2002–2007
Inference loop	Balanced	10^4	2101
Small stress	Balanced, high-cancel, replace-heavy, deep-book, wide-price-range, bursty, long-same-symbol, adversarial-lifecycle	4000–8000	20260531–20260538

C. Recorded-L2 Replay Trace

The raw fixture under `data/samples/` is `binance-depth-sample.raw.jsonl`. Its normalised CSV and binary forms contain 11 logical events: two adds, two cancels, one replace and six snapshot records. Table 8 reports the normalisation checks and replay outputs. Equal CSV and binary event checksums show logical parity for this fixture. The diagnostics and execution-report checksums equal the FNV offset basis because those streams contain no entries in this replay.

Table 8. Expected normalisation and replay values from `data/samples/binance_depth_sample.expected.json`. Checksums identify fixture outputs, not implementation equivalence.

Surface	Expected value
Normalised event count	11
CSV event checksum	9673906315134520083
Binary event checksum	9673906315134520083
Diagnostic warnings / errors	0 / 0
Diagnostics checksum	14695981039346656037
Execution-report checksum	14695981039346656037
Final book checksum	2539005926052284398
Sequence valid	true

D. Reproducibility

The source and machine-readable results are at <https://github.com/anannyenaik/asterion>. This paper describes release v0.1.0 at commit `3a764d4b7ba1075e64a416b07b4de12f5b7df1c3`. The Hamilton8 measurements were produced from source commit `216f473667aede486c95fe9607e09646932ab5a1`, as recorded in the result manifest, so the release commit is distinct from the commit that generated these timings. The four empirical vector figures and all table values are taken unchanged from the v0.1.0 checkout; Figure 3 uses copies of two of those plots with added panel letters.

From the Asterion repository root, build and run the source checks with:

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release \
  -DASTERION_BUILD_PYTHON=ON
cmake --build build
ctest --test-dir build --output-on-failure
PYTHONPATH=build/python python -m pytest python/tests
```

Regenerate and check the empirical vector figures with:

```
pip install -e "[figures]"
python figures/generate_figures.py
python figures/generate_figures.py --check
```

Figure 1 is drawn in TikZ from `figures/design.tex` in the paper source. Compile the paper with:

```
latexmk -pdf -jobname=Asterion main.tex
```